Tara Guardrail Implementation Summary

This document describes the project's actual implementation model: **bounded execution by policy and guardrails**, not an unrestricted or "no guardrails" mode.

> **Core principle:** Tara is built as an allowlist-bound agent. It is designed to operate only within configured lab or competition scope, and it fails closed when the target, destination, or action is not explicitly allowed.

1. Scope policy is the first gate

In `core/scope.py`, `ScopePolicy` validates targets before any network action runs. It checks:

- allowed hosts
- allowed CIDR ranges
- allowed ports
- allowed filesystem roots
- special-danger ranges such as link-local, metadata, CGNAT, multicast, and reserved space
- DNS-resolved addresses to block rebinding and name-based bypasses
- explicit onion target restrictions

If a hostname resolves to an out-of-scope IP, the check raises `ScopeViolation` and the tool is refused.

2. Socket-level egress guard is the second gate

In `core/netguard.py`, `NetworkGuard.install()` monkey-patches socket connection logic so every outbound connection is examined. It allows:

- loopback
- approved control-plane hosts
- targets within the configured scope

Everything else raises an error and blocks the connection before it reaches the network.

3. Tool execution is centralized

In `core/toolrunner.py`, all tool calls pass through `ToolRunner.run()`. Before execution, the runner:

- looks up the tool
- asks the tool which targets it intends to touch
- runs those targets through the scope policy
- refuses the action if it is out of scope
- optionally logs a dry-run instead of executing
- enforces a timeout so a stuck tool cannot stall the loop

4. Defense layer enforces confidence before action

In `defense/assurance.py` and `defense/defense.py`, the project does not act on a single noisy signal. It uses:

- deduplication to collapse repeated findings
- confidence scoring to rank detection quality
- confirmation logic to replay or validate suspicious requests
- false-positive guards so block rules are not applied to checker or benign traffic
- flag validation to reject malformed or hallucinated tokens

5. Mitigation is policy-bound and reversible

In `defense/mitigation.py`, mitigation is represented as a structured rule object, not a wild firewall change. The code:

- creates explicit block rules
- prefers precise payload filtering over broad source blocking
- uses guards to reject unsafe patterns
- supports emit-only enforcement unless the operator explicitly opts in

6. Offense is review-driven by default

In `offense/offense.py`, captured flags are queued for review unless `auto_submit` is explicitly enabled. This prevents low-confidence or hallucinated flags from being sent automatically.

7. Config values that define the safety wall

In `config/default.yaml`, the enforcement values include:

- `scope.allowed_cidrs` — explicit in-scope ranges

- `scope.allowed_hosts` — explicit named hosts
- `scope.allowed_ports` — allowed ports
- `scope.block_special: true` — blocks dangerous non-target ranges
- `scope.resolve_check: true` — validates every resolved address
- `scope.enforce_egress: true` — installs socket-level guard
- `defense.auto_mitigate: false` — mitigations require explicit opt-in
- `offense.auto_submit: false` — flags require review by default

Bottom line: Tara’s design is not “no guardrails.” It is a layered, fail-closed system with explicit policy, egress filtering, execution gating, detection scoring, and review-based action control.